Smart Bank and ATM Simulation System

Detailed Project Report and Specification

Course: Object-Oriented Programming using Java

Project Type: Group Project (4-5 members)

Technology: Java SE, Swing GUI, File I/O / JDBC

Document Contents: Aim, scope, requirements, OOP concepts, class design, business logic, data design, UI specification, flow charts, exceptions, testing and project plan

Index

1. Introduction . . . 3
- 1.1 Project Overview . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Aim . . . 3
- 1.4 Objectives . . . 3
- 1.5 Expected Outcome . . . 3

2. Scope of the Project . . . 3
- 2.1 In Scope . . . 3
- 2.2 Out of Scope . . . 3
- 2.3 Assumptions and Constraints . . . 3

3. Requirements Analysis . . . 4
- 3.1 User Roles . . . 4
- 3.2 Functional Requirements . . . 4
- 3.3 Non-Functional Requirements . . . 4

4. Object-Oriented Concepts Used . . . 4
- 4.1 Code Sketches . . . 5

5. Class Design . . . 5
- 5.1 Inheritance Structure . . . 5
- 5.2 Detailed Class Description . . . 5
- 5.3 Relationships Between Classes . . . 6

6. Business Logic . . . 6
- 6.1 System Parameters . . . 6
- 6.2 Identifier Generation . . . 6
- 6.3 Account Opening Rules . . . 6
- 6.4 Deposit Rules . . . 6
- 6.5 Withdrawal Rules . . . 7
- 6.6 Fund Transfer Rules . . . 7
- 6.7 Interest and Monthly Charges . . . 7
- 6.8 Card and PIN Security Rules . . . 7
- 6.9 Admin Rules . . . 7
- 6.10 Validation Summary . . . 8

7. Data Storage Design . . . 8
- 7.1 Files / Database Tables . . . 8
- 7.2 Persistence Approach . . . 8

8. User Interface Specifications . . . 8
- 8.1 Screens . . . 8
- 8.2 Interface Guidelines . . . 9

9. System Flow Charts . . . 10
- 9.1 Overall System Flow . . . 10
- 9.2 ATM Withdrawal Flow . . . 11

10. Exception Handling . . . 11

11. Testing Plan . . . 12

12. Module Division, Project Plan and Conclusion . . . 12
- 12.1 Module Division . . . 12
- 12.2 Timeline . . . 12
- 12.3 Future Enhancements . . . 12
- 12.4 Conclusion . . . 12

1. Introduction

1.1 Project Overview

The Smart Bank and ATM Simulation System is a desktop application written in Java that models the day-to-day working of a retail bank and its Automated Teller Machine (ATM). It allows a customer to hold one or more accounts, deposit and withdraw money, transfer funds, check balances and view past transactions. It also allows a bank administrator to open and close accounts, unblock cards and generate reports. The ATM component reproduces the real-world experience of inserting a card, entering a PIN and choosing a service, including security rules such as locking the card after repeated wrong PIN entries.

The project is deliberately designed around object-oriented programming. Every real-world entity (customer, account, card, ATM, transaction) is modelled as a class, and the rules that govern how these entities behave are captured as methods inside those classes. This keeps the business logic in one place, makes the code easy to test and allows new account types or services to be added without rewriting existing code.

1.2 Problem Statement

Manual banking records are slow, error-prone and hard to audit. Students learning Java often study OOP concepts in isolation and rarely see them working together in a realistic system. This project addresses both issues by building a complete, rule-driven banking simulation in which inheritance, polymorphism, abstraction, encapsulation, exception handling and persistence all have a natural and necessary role.

1.3 Aim

To design and develop a simple, secure and well-structured banking and ATM simulation in Java that demonstrates how object-oriented programming is used to model real banking operations and enforce banking business rules.

1.4 Objectives

- Model customers, accounts, cards, ATMs and transactions as cooperating classes with clear responsibilities.
- Implement two account types (Savings and Current) whose withdrawal and interest rules differ, using polymorphism.
- Enforce security rules: PIN validation, limited attempts, card blocking and session time-out.
- Enforce financial rules: minimum balance, overdraft limit, per-transaction and daily limits, and atomic fund transfers.
- Keep a permanent, auditable record of every transaction and provide mini statements and admin reports.
- Persist all data so the system state survives restarts, using file handling or JDBC.
- Provide a clear Swing graphical interface for customers, ATM users and administrators.

1.5 Expected Outcome

The team will deliver a working Java application, tested classes covering all business rules in this report, a persistent data store, and this report documenting the design, rules and test results.

2. Scope of the Project

2.1 In Scope

- Role-based login for Customer and Admin users.
- Creation and closure of Savings and Current accounts with validation of customer details.
- Deposit, withdrawal, fund transfer, balance enquiry, PIN change and mini statement (last 10 transactions).
- ATM simulation: card and PIN authentication, cash dispensing with limits, receipt generation, session time-out.
- Monthly processing: interest credit for Savings accounts, maintenance and overdraft charges for Current accounts.
- Admin functions: unblock card, view customers, view all transactions, reports (daily summary, blocked cards, low balance accounts).
- Data persistence and audit logging of every financial and administrative action.

2.2 Out of Scope

- Connection to real banks, payment gateways, SMS or e-mail services.
- Real cash-handling hardware, card readers or biometric devices.
- Loans, fixed deposits, credit cards, cheque processing and foreign currency.
- Online or mobile banking; the system is a desktop simulation only.

2.3 Assumptions and Constraints

- All amounts are in a single currency (Rs.) and are handled using BigDecimal or whole rupees to avoid floating-point errors.
- The application runs on a single machine; there is no concurrent access from multiple computers.
- Business parameters (limits, rates, charges) are defined in one configuration class so they can be changed without touching other code.
- The system date and time of the computer are used for all timestamps.

3. Requirements Analysis

3.1 User Roles

Role	Description	Main Capabilities
Customer	Account holder who uses the dashboard or the ATM.	View balance, deposit, withdraw, transfer, change PIN, view mini statement.
ATM User	A customer interacting through the ATM screen using card and PIN.	Withdraw cash, check balance, change PIN, print receipt.
Admin	Bank employee with a separate login.	Create and close accounts, unblock cards, view reports and audit log.

3.2 Functional Requirements

ID	Requirement
FR1	The system shall authenticate customers and admins using ID and password or card number and PIN.
FR2	The system shall allow an admin to create Savings and Current accounts after validating the customer's details and minimum opening deposit.
FR3	The system shall allow deposits to active accounts within the single-deposit limit.
FR4	The system shall allow withdrawals only if the account rules, per-transaction limit, daily limit and ATM cash availability are all satisfied.
FR5	The system shall transfer funds between two active accounts as a single all-or-nothing operation.
FR6	The system shall block a card after three consecutive wrong PIN entries and allow only an admin to unblock it.
FR7	The system shall record every transaction with a unique ID, type, amount, date-time, status and balance after the transaction.
FR8	The system shall calculate and credit monthly interest to Savings accounts and apply monthly charges to Current accounts.
FR9	The system shall generate mini statements, receipts and admin reports.
FR10	The system shall save all data to persistent storage after each successful operation and reload it at start-up.

3.3 Non-Functional Requirements

Quality	Requirement
Security	PINs and passwords are stored only as SHA-256 hashes; never printed or logged; card blocked after 3 failures; ATM session ends after 60 seconds of inactivity.
Reliability	No operation may leave data half-updated; failed transfers are rolled back.
Usability	Every screen shows clear messages for success and error; amount fields accept only valid numeric input.
Maintainability	Business rules live inside model classes and a Config class; UI classes contain no business logic.
Performance	Any single operation completes in under one second for up to 10,000 accounts.
Portability	Runs on any operating system with Java 11 or later.

4. Object-Oriented Concepts Used

The project uses all the core OOP concepts. The table below states where each concept appears and why it is the right choice, followed by short code sketches of the most important ones.

Concept	Where it is used	Reason
Encapsulation	balance, pin, password, failedAttempts and blocked are private fields in Account, Card and User.	Prevents outside code from changing a balance or PIN without going through validated methods such as deposit() and validatePin().
Inheritance	Customer and Admin extend User; SavingsAccount and CurrentAccount extend Account.	Shared fields and behaviour are written once in the parent class and reused.
Polymorphism	withdraw() and calculateInterest() are overridden in each account type. A List<Account> is processed without knowing the exact type.	Lets the Bank class apply the correct rule for each account automatically (runtime polymorphism).
Abstraction	User and Account are abstract classes; Transactable and Printable are interfaces.	Defines what an account must do without fixing how; prevents creating a meaningless generic account.
Interfaces	Transactable (deposit, withdraw, transfer), Printable (printReceipt, printStatement).	Gives unrelated classes (Account, Transaction, ATM) a common capability contract.
Exception handling	Custom checked exceptions for business rule violations.	Separates error handling from normal logic and lets the UI show precise messages.
Collections	ArrayList for transactions, HashMap<String, Account> for fast account lookup.	Efficient storage and retrieval by account number.
File I/O / JDBC	FileManager (or DBManager) saves and loads all objects.	Provides persistence between program runs.
Static and final	Config constants (limits, rates) and ID generator counters.	One shared source for rules; constants cannot be modified accidentally.

4.1 Code Sketches

```
public abstract class Account implements Transactable, Printable {
    private final String accountNumber;
    private BigDecimal balance;                 // encapsulated
    protected final List<Transaction> history = new ArrayList<>();

    public void deposit(BigDecimal amt) throws InvalidAmountException { ... }
    public abstract void withdraw(BigDecimal amt) throws InsufficientFundsException;
    public abstract BigDecimal calculateInterest();   // polymorphic
}

public class SavingsAccount extends Account {
    @Override public void withdraw(BigDecimal amt) throws InsufficientFundsException {
        if (getBalance().subtract(amt).compareTo(Config.MIN_BALANCE) < 0)
            throw new InsufficientFundsException("Minimum balance must be maintained");
        debit(amt);
    }
    @Override public BigDecimal calculateInterest() {
        return getBalance().multiply(Config.SAVINGS_RATE)
                .divide(new BigDecimal("1200"), 2, RoundingMode.HALF_UP);
    }
}
```

5. Class Design

5.1 Inheritance Structure

The diagram shows the two inheritance hierarchies. Hollow arrows point from the child class to its parent class. Names in italics-style labels such as (abstract) mean the class cannot be instantiated directly.

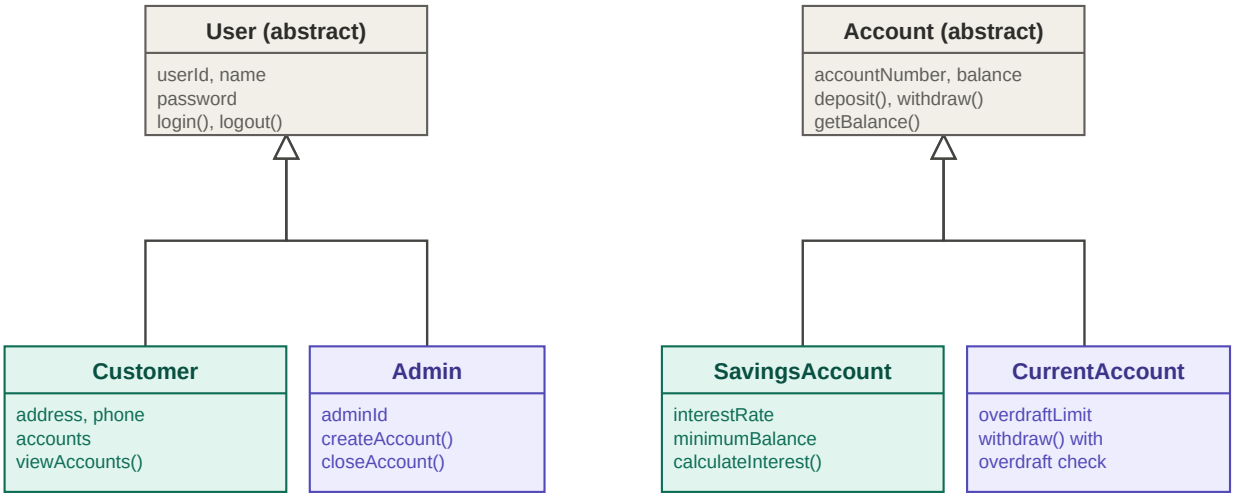

Figure 1: Inheritance hierarchies of the system

5.2 Detailed Class Description

Class	Properties	Behaviour (methods)
User (abstract)	userId, name, passwordHash	login(id, password), logout(), changePassword()
Customer	address, phone, dateOfBirth, List<Account> accounts	viewAccounts(), requestTransfer(), getMiniStatement()
Admin	adminId, role	createAccount(), closeAccount(), unblockCard(), generateReport()
Account (abstract)	accountNumber, balance, owner, openDate, status, history	deposit(), withdraw() [abstract], calculateInterest() [abstract], getBalance(), addTransaction()
SavingsAccount	interestRate, minimumBalance	withdraw() with minimum balance rule, calculateInterest(), creditMonthlyInterest()
CurrentAccount	overdraftLimit, monthlyCharge	withdraw() with overdraft rule, applyMonthlyCharges()
Transaction	transactionId, accountNumber, type, amount, dateTime, status, balanceAfter, referenceId	getDetails(), toCsvLine()
Card	cardNumber, pinHash, expiryDate, failedAttempts, blocked, accountNumber	validatePin(), block(), unblock(), changePin()
ATM	atmId, location, cashAvailable, currentSession	authenticate(), dispenseCash(), checkBalance(), printReceipt(), endSession()

Class	Properties	Behaviour (methods)
Bank	bankName, Map<String,Account> accounts, Map<String,Customer> customers, Map<String,Card> cards	openAccount(), closeAccount(), findAccount(), transfer(), runMonthEnd()
Config	all limits, rates and charges (static final)	constants only
FileManager / DBManager	filePaths or connection	saveAll(), loadAll(), appendTransaction()
Validator	none (static utility)	isValidPhone(), isValidPin(), isValidAmount(), isAdult()
IdGenerator	static counters	nextCustomerId(), nextAccountNumber(), nextTransactionId()

5.3 Relationships Between Classes

Relationship	Type	Meaning
Customer - Account	Association (1 to many)	A customer can hold several accounts; each account has exactly one primary owner.
Account - Transaction	Composition (1 to many)	Transactions belong to one account; they have no meaning without it.
Account - Card	Association (1 to 1)	Each account has one debit card used at the ATM.
Bank - Customer/Account/Card	Aggregation	The Bank class holds and manages all customers, accounts and cards.
ATM - Bank	Dependency	The ATM calls Bank methods to authenticate and to move money.
Account - Exceptions	Dependency	Account methods throw custom exceptions when a rule is violated.

6. Business Logic

This section is the heart of the system. It lists every rule the program enforces. All numeric parameters are defined once in the Config class so that they can be changed in one place. The values below are the defaults chosen for the project.

6.1 System Parameters

Parameter	Value	Parameter	Value
Minimum opening deposit - Savings	Rs. 1,000	ATM per-transaction limit	Rs. 10,000
Minimum opening deposit - Current	Rs. 5,000	ATM daily withdrawal limit	Rs. 20,000
Minimum balance - Savings	Rs. 1,000	Withdrawal denomination	Multiples of Rs. 100
Savings interest rate	4% per year	Daily transfer limit	Rs. 50,000
Overdraft limit - Current	Rs. 10,000	Single cash deposit limit	Rs. 1,00,000
Overdraft charge	1.5% per month on amount used	PIN length / wrong attempts allowed	4 digits / 3
Current account monthly charge	Rs. 100	ATM session time-out	60 seconds
Minimum customer age	18 years	Card validity	5 years from issue

6.2 Identifier Generation

Item	Format	Example
Customer ID	C + 4-digit running number	C1001
Account number	10-digit running number starting at 1000000001	1000000001
Card number	16 digits, first 6 fixed as bank code, last digit check digit	5234 5600 0000 0017
Transaction ID	TXN + date (yyyyMMdd) + 5-digit daily sequence	TXN20261010000042

6.3 Account Opening Rules

1. Admin enters customer name, date of birth, phone, address and chooses the account type.
2. The Validator checks: name is not empty and has only letters and spaces; phone is 10 digits and not already registered; the customer is at least 18 years old.
3. The opening deposit must be at least the minimum for the chosen account type (Rs. 1,000 for Savings, Rs. 5,000 for Current).
4. The customer chooses a 4-digit PIN. It must not be all the same digit (1111) or a simple sequence (1234, 4321). The PIN is stored only as a SHA-256 hash.
5. The system generates the Customer ID, account number and card number, sets the account status to ACTIVE, records the opening deposit as the first transaction and saves the data.

6.4 Deposit Rules

- The account must be ACTIVE. Deposits to CLOSED or FROZEN accounts are rejected.

- The amount must be greater than zero and not more than Rs. 1,00,000 in a single deposit.
- New balance = old balance + amount. A DEPOSIT transaction with status SUCCESS and the new balance is recorded.
- For a Current account that is overdrawn, the deposit first reduces the overdrawn amount.

6.5 Withdrawal Rules

A withdrawal passes through a chain of checks. If any check fails, the operation stops, no balance is changed and a FAILED transaction is logged with the reason. Figure 3 in Section 9 shows the same sequence as a flow chart.

1. The card must not be blocked and the account must be ACTIVE.
2. The amount must be a positive multiple of Rs. 100.
3. The amount must not exceed Rs. 10,000 per transaction.
4. Today's withdrawals plus this amount must not exceed Rs. 20,000.
5. The ATM must hold at least this much cash.
6. The account type rule must pass (this step is polymorphic):

Account type	Available amount	Rule
Savings	balance - Rs. 1,000	After withdrawal the balance must remain at least Rs. 1,000.
Current	balance + Rs. 10,000	After withdrawal the balance may go negative but not below -Rs. 10,000.

Worked examples. (a) Savings balance Rs. 8,000, request Rs. 7,500: remaining Rs. 500 is below Rs. 1,000, so InsufficientFundsException is thrown. Request Rs. 7,000 is allowed and leaves Rs. 1,000. (b) Current balance Rs. 3,000, request Rs. 9,000: new balance is -Rs. 6,000, which is within the overdraft limit, so it is allowed; the overdraft charge at month end is 1.5% of Rs. 6,000 = Rs. 90.

6.6 Fund Transfer Rules

- Source and destination accounts must be different and both must be ACTIVE.
- The amount must be positive and total transfers from the source account on that day must not exceed Rs. 50,000.
- The source account rule (minimum balance or overdraft) is applied exactly as for a withdrawal. ATM cash limits do not apply to transfers.
- The transfer is atomic: the source is debited first, then the destination is credited. If the credit fails for any reason, the debit is reversed and the transfer is marked FAILED.
- Two transactions are written, TRANSFER_OUT and TRANSFER_IN, sharing the same referenceId so that they can be traced together.

```
public void transfer(String from, String to, BigDecimal amt) throws BankException {
    Account src = findAccount(from), dst = findAccount(to);
    validateTransfer(src, dst, amt);          // active, different, daily limit
    src.withdraw(amt);                        // polymorphic rule check + debit
    try { dst.deposit(amt); }
    catch (Exception e) { src.reverseDebit(amt); throw new TransferFailedException(e); }
    logTransfer(src, dst, amt);
}
```

6.7 Interest and Monthly Charges

On the last day of every month the Bank class loops through all accounts and calls the polymorphic month-end method of each one. Savings accounts earn interest; Current accounts pay charges.

Account	Calculation	Example
Savings interest	interest = month-end balance x rate / 100 / 12, rounded to 2 decimals and credited as an INTEREST transaction.	Balance Rs. 50,000 at 4%: 50,000 x 4 / 100 / 12 = Rs. 166.67
Current monthly charge	Flat Rs. 100 debited as a CHARGE transaction.	Rs. 100 every month
Overdraft charge	1.5% of the overdrawn amount at month end.	Overdrawn Rs. 6,000: Rs. 90

6.8 Card and PIN Security Rules

- PIN is exactly 4 digits and is compared by hash, never in plain text.
- Each wrong PIN increments failedAttempts. A correct PIN resets it to 0. On the third consecutive failure the card is blocked and the message "Card blocked, contact bank" is shown.
- A blocked card cannot be used until an admin unblocks it; the unblock action is written to the audit log.
- PIN change requires the old PIN, and the new PIN must pass the same strength rules as in 6.3 and must differ from the old PIN.
- If there is no user action for 60 seconds, the ATM session ends automatically and the welcome screen is shown (Swing Timer).
- An expired card (past expiry date) is rejected at authentication.

6.9 Admin Rules

- An account can be closed only when its balance is exactly zero; otherwise the admin must first pay out the remaining balance.
- Closing an account also deactivates its card; closed accounts stay in the data for audit but accept no transactions.
- Every admin action (create, close, unblock, report) is logged with admin ID and time.
- Reports available: daily transaction summary, list of blocked cards, accounts below minimum balance, customer list.

6.10 Validation Summary

Field	Validation rule	Error message shown
Amount	Numeric, greater than 0, at most 2 decimals	Please enter a valid amount
Phone	Exactly 10 digits, unique	Invalid or already registered phone number
Date of birth	Customer is 18 or older	Customer must be at least 18 years old
PIN	4 digits, not repeated or sequential	PIN is too simple or invalid
Account number	10 digits and exists	Account not found
Transfer target	Different from source and ACTIVE	Invalid destination account

7. Data Storage Design

All objects are saved by the FileManager class (or DBManager when JDBC is used). Data is loaded into memory when the program starts, kept in HashMaps and ArrayLists while it runs, and written back after every successful operation so that a crash loses at most the operation in progress. Transactions are appended to their file and never edited, which gives a permanent audit trail.

7.1 Files / Database Tables

File / Table	Columns
customers	customerId, name, dateOfBirth, phone, address, passwordHash
accounts	accountNumber, customerId, type (SAVINGS/CURRENT), balance, openDate, status
cards	cardNumber, accountNumber, pinHash, expiryDate, failedAttempts, blocked
transactions	transactionId, accountNumber, type, amount, dateTime, status, balanceAfter, referenceId
admins	adminId, name, passwordHash
audit_log	logId, adminId, action, targetId, dateTime

7.2 Persistence Approach

- **File option:** comma-separated text files, one per table, read with BufferedReader and written with BufferedWriter. Simple and needs no setup.
- **Database option:** the same tables created in MySQL or SQLite and accessed with JDBC PreparedStatement, which also protects against SQL injection.
- A copy of each data file is written as a backup before every save, so a corrupted file can be recovered.
- Loading and saving are hidden behind the FileManager interface so the rest of the program does not depend on which option is used.

8. User Interface Specifications

The interface is built with Java Swing. Each screen is a JFrame or JPanel in its own class; screens call model classes and display results but contain no business rules. A common colour theme and font are used for consistency, forms use GridBagLayout, and the main window uses CardLayout to switch between screens.

8.1 Screens

Screen	Components	Actions and messages
Login	Role selector (Customer / Admin), ID field, password field, Login and Exit buttons.	Validates input; shows "Invalid ID or password" on failure.
Customer Dashboard	Welcome label, account selector, balance label, buttons: Deposit, Withdraw, Transfer, Mini Statement, Change PIN, Logout.	Balance refreshes after every operation.
Deposit / Withdraw dialog	Amount field, Confirm and Cancel buttons.	Shows rule errors such as limit exceeded or minimum balance.
Transfer screen	Destination account field, amount field, Transfer button.	Shows confirmation with destination name before executing.
Mini Statement	JTable with date, type, amount, balance (last 10), Print button.	Print creates a text receipt.
ATM Welcome / PIN	Card number field, on-screen numeric keypad, masked PIN field, Enter and Clear buttons.	Shows attempts left; "Card blocked" after 3 failures.

Screen	Components	Actions and messages
ATM Menu	Withdraw, Balance Enquiry, Change PIN, Mini Statement, Exit; quick amounts 500 / 1000 / 2000 / 5000 / Other.	Session timer resets on each action; 60-second time-out.
Admin Panel	Tabbed pane: Create Account form, Close Account, Customers table, Transactions table, Blocked Cards with Unblock button, Reports.	Validation messages for each field; confirmation before closing.

8.2 Interface Guidelines

- Amount fields accept digits only (input filter) and show the currency label Rs.
- PIN and password fields are always masked.
- Every destructive action (close account, transfer) asks for confirmation.
- Errors are shown with JOptionPane dialogs using the exact messages in the validation table.
- Windows use fixed minimum sizes and consistent button placement (primary action at bottom right).

9. System Flow Charts

9.1 Overall System Flow

Figure 2 shows the complete flow of the application. After login and role check, customers (teal) and admins (purple) follow separate paths. Every path validates the request, updates data, saves it and ends with logout. A wrong PIN lets the user retry until the third failure, which locks the card.

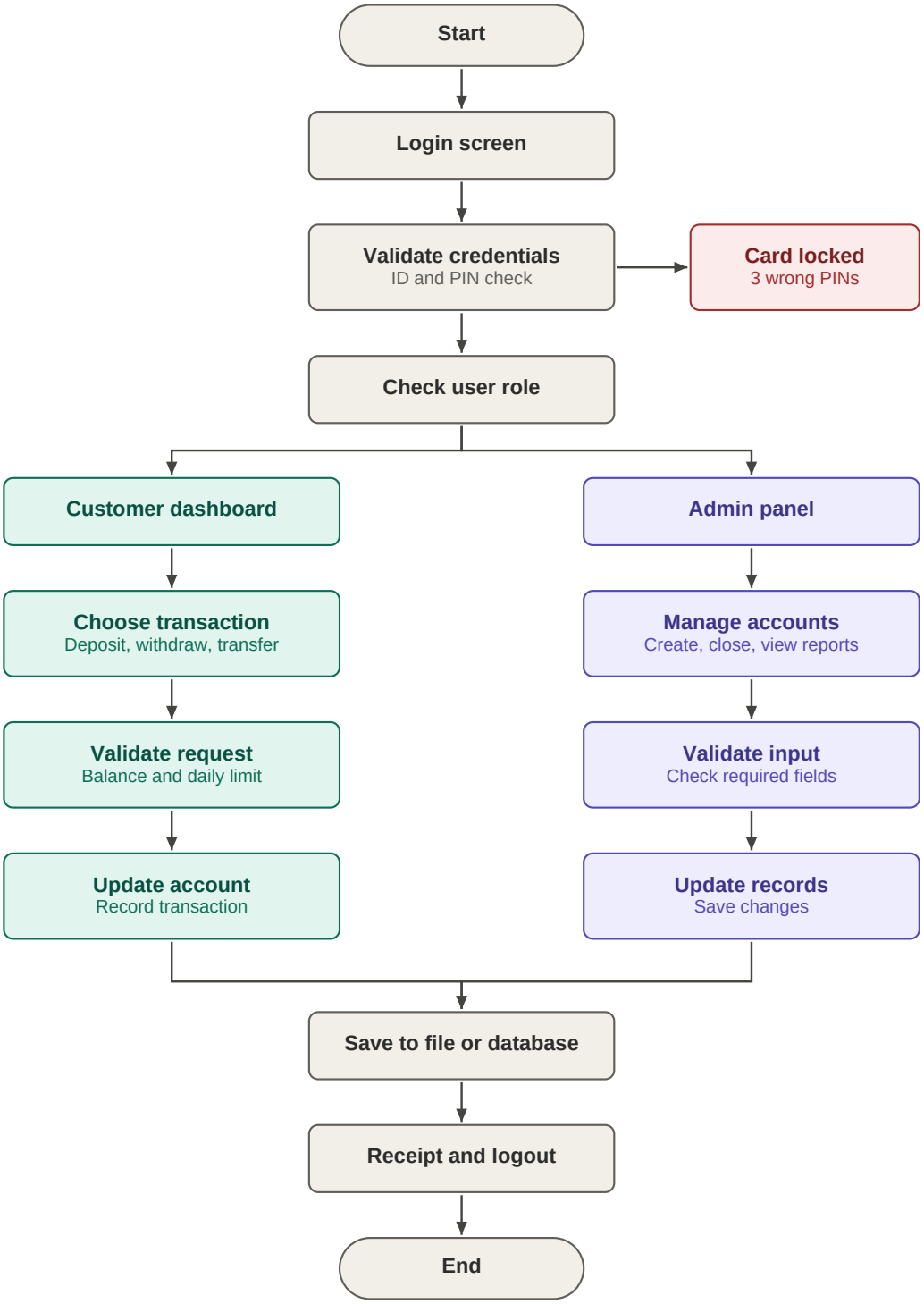

Figure 2: Overall flow of the Smart Bank and ATM system

9.2 ATM Withdrawal Flow

Figure 3 expands the most rule-heavy operation. Each diamond is a business rule from Section 6.5 and each red box is the exception that is raised when the rule fails. When any rule fails the transaction is logged as FAILED and the balance stays unchanged.

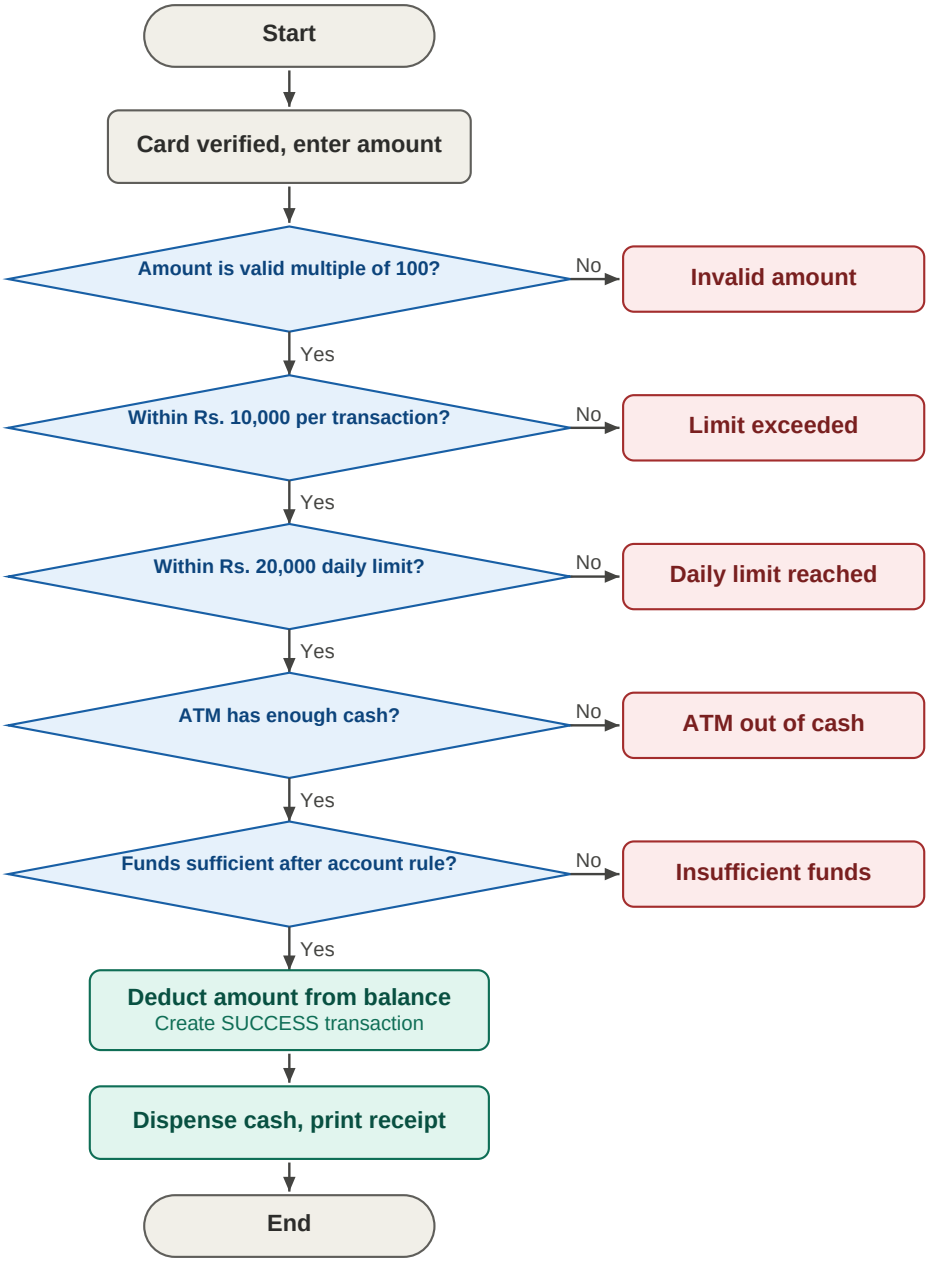

Figure 3: Decision flow of an ATM cash withdrawal

10. Exception Handling

Business rule violations are reported through custom checked exceptions that extend a common BankException. The model classes throw them; the UI classes catch them and show the message in a dialog. Unexpected errors (for example a file that cannot be read) are caught at the top level, logged and shown as a generic message so that the program never crashes.

Exception	Thrown when	Message to user
InvalidAmountException	Amount is zero, negative, not a multiple of 100 or has too many decimals.	Please enter a valid amount
InsufficientFundsException	Minimum balance or overdraft rule would be broken.	Insufficient funds
LimitExceededException	Per-transaction, daily withdrawal or daily transfer limit is exceeded.	Transaction limit exceeded
InvalidPinException	Entered PIN does not match; attempts left are shown.	Incorrect PIN, 2 attempts left
AccountLockedException	Card is blocked after 3 wrong attempts.	Card blocked, contact bank
AccountNotFoundException	Account or card number does not exist.	Account not found
AccountInactiveException	Account is closed or frozen.	Account is not active
ATMOutOfCashException	ATM cash is lower than requested amount.	ATM cannot dispense this amount

Exception	Thrown when	Message to user
TransferFailedException	Credit to destination fails and debit is reversed.	Transfer failed, no money deducted
DataAccessException	Data file or database cannot be read or written.	System error, please try later

11. Testing Plan

Each business rule is verified with unit tests on the model classes (JUnit) and with manual test cases through the interface. Boundary values are tested on both sides of every limit.

ID	Scenario	Input	Expected result
T1	Valid login	Correct ID and password	Dashboard opens
T2	Wrong PIN three times	Wrong PIN x 3	Card blocked; further attempts rejected
T3	Deposit valid amount	Rs. 5,000	Balance increases by Rs. 5,000; transaction logged
T4	Deposit above limit	Rs. 1,50,000	Rejected with limit message
T5	Savings withdrawal below minimum	Balance 8,000; withdraw 7,500	InsufficientFundsException; balance unchanged
T6	Savings withdrawal to minimum	Balance 8,000; withdraw 7,000	Allowed; balance Rs. 1,000
T7	Current account overdraft	Balance 3,000; withdraw 9,000	Allowed; balance -Rs. 6,000
T8	Not a multiple of 100	Withdraw Rs. 550	InvalidAmountException
T9	Daily limit	Withdraw 10,000 twice, then 500	Third request rejected
T10	Transfer success	Rs. 2,000 between two active accounts	Both balances updated; two linked transactions
T11	Transfer to same account	Source = destination	Rejected
T12	Transfer rollback	Destination inactive	Source balance restored; status FAILED
T13	Monthly interest	Savings Rs. 50,000	Rs. 166.67 credited
T14	Close account with balance	Balance Rs. 100	Rejected until balance is zero
T15	Restart persistence	Close and reopen program	All balances and history unchanged

12. Module Division, Project Plan and Conclusion

12.1 Module Division

Member	Module	Classes and screens
Member 1	Users and security	User, Customer, Admin, Card, Validator; Login screen
Member 2	Accounts and rules	Account, SavingsAccount, CurrentAccount, Config, interest and charge logic
Member 3	Transactions and exceptions	Transaction, IdGenerator, all custom exceptions, mini statement
Member 4	Bank and ATM logic	Bank, ATM, transfer logic, session time-out, month-end processing
Member 5	Interface and persistence	Swing screens, FileManager or DBManager, reports, testing support

For a group of four, Member 5's work is shared between Members 3 and 4.

12.2 Timeline

Phase	Activity	Duration
1	Requirement study, class design, specification document	Week 1
2	Core classes: users, accounts, transactions, exceptions	Week 2
3	Business logic: withdrawal, transfer, interest, security rules	Week 3
4	Swing interface and persistence	Week 4
5	Integration, testing, bug fixing, final report and demonstration	Week 5

12.3 Future Enhancements

- Loan and fixed deposit modules with their own account classes.
- E-mail or SMS style notifications for transactions and low balance.
- Multi-user network version with a central database server.
- Graphical analytics for admins (daily transactions chart).

12.4 Conclusion

The Smart Bank and ATM Simulation System brings together every major concept of object-oriented programming in a realistic setting. The detailed business rules ensure that the application behaves like a real banking system, while the layered class design keeps the code clean, testable and easy to extend. The project gives the team practical experience in analysis, design, implementation, testing and documentation of a complete Java application.